

Virtualisasi: satu perangkat, banyak lingkungan

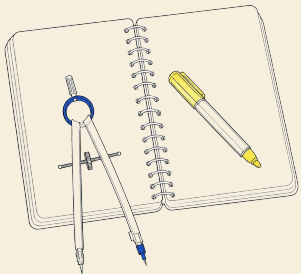
Layanan dan Sistem Virtual (STI5134) • Pertemuan 4

Memahami empat bentuk utama virtualisasi dan menentukan penerapannya sesuai kebutuhan sistem.



Tujuan hari ini: pahami konsep dan pilih bentuknya

Capaian pembelajaran yang ditargetkan setelah pertemuan ini selesai:



- 1 **Jelaskan prinsip virtualisasi** serta alasan ekonomis dan teknis mengapa organisasi mengadopsinya.
- 2 **Uraikan empat tipe utama:** Desktop, Application, Server, dan Hardware Virtualization secara sistematis.
- 3 **Bandingkan manfaat dan batasannya**, termasuk trade-off performa, kompleksitas, serta ketergantungan jaringan.
- 4 **Pilih pendekatan yang sesuai** saat dihadapkan pada skenario studi kasus infrastruktur nyata.

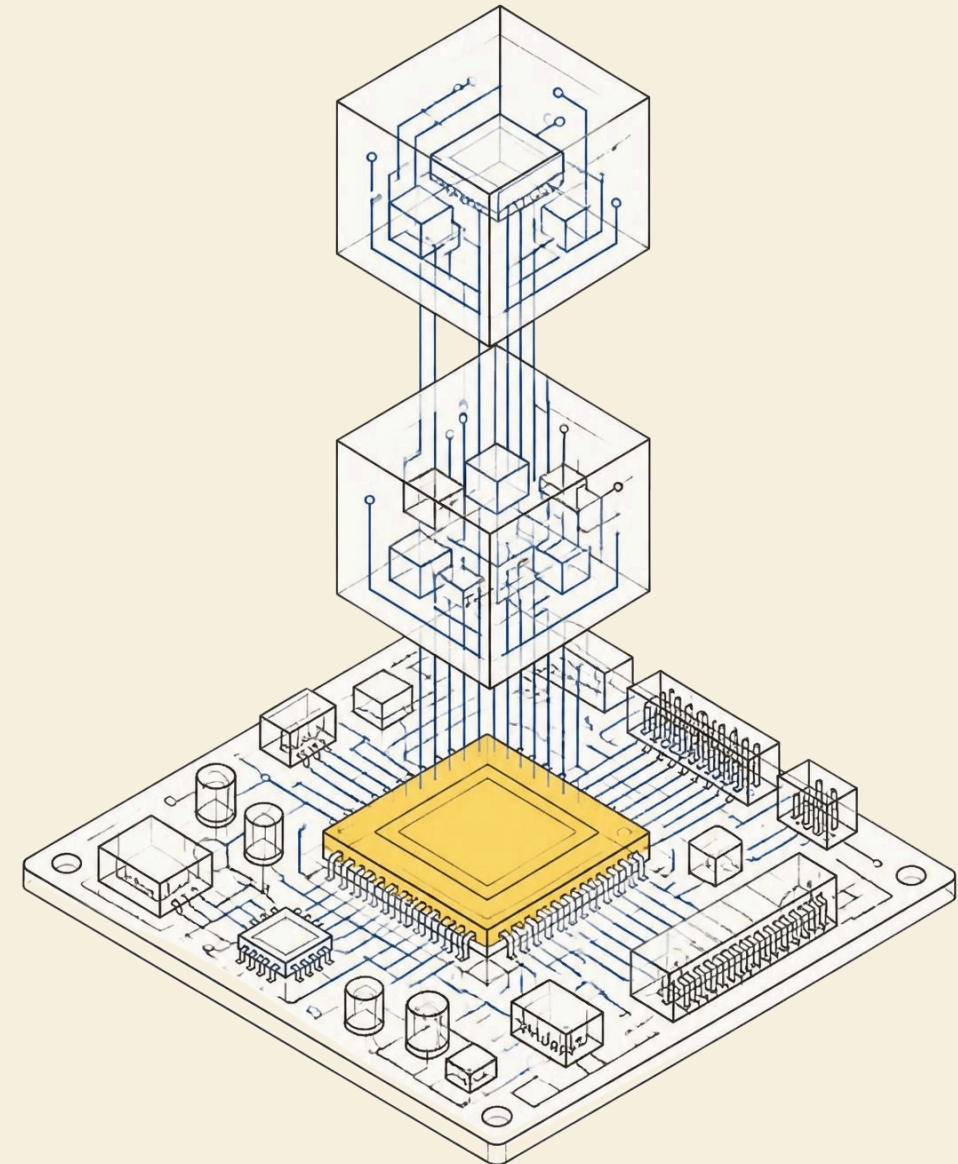
Virtualisasi memisahkan lingkungan logis dari perangkat fisik

Konsep Kunci

Virtualisasi adalah lapisan perangkat lunak yang mengabstraksi sumber daya komputasi fisik sehingga dapat dibagi menjadi beberapa lingkungan virtual yang saling terisolasi.

Contoh praktis: satu unit laptop dapat menjalankan Windows sebagai sistem operasi utama (host) sekaligus menjalankan Ubuntu sebagai lingkungan tamu (guest) di dalam VirtualBox secara bersamaan.

Prinsip inti: satu perangkat keras fisik mampu menampung beragam lingkungan komputasi independen.



Virtualisasi membuat satu infrastruktur melayani banyak lingkungan



Abstraksi logis

Memisahkan perangkat fisik (CPU, RAM, storage, network) menjadi kumpulan sumber daya virtual yang fleksibel, terukur, dan dapat dialokasikan dinamis.



Rantai komputasi

Mendefinisikan interaksi berjenjang antara hardware fisik, lapisan hypervisor, virtual machine mandiri, hingga runtime aplikasi pengguna.



Batas isolasi

Membedakan virtualisasi mesin utuh (emulasi hardware dan kernel terpisah) dengan isolasi proses tingkat OS yang lebih ringan dan efisien.

Empat tipe virtualisasi menjawab empat kebutuhan berbeda

Setiap bentuk virtualisasi mengisolasi komponen komputasi pada lapisan yang berbeda sesuai tujuan operasional.



Desktop

Memusatkan lingkungan kerja pengguna di server agar mudah diakses dari perangkat tipis.



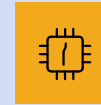
Application

Mengisolasi aplikasi dan dependensinya dari sistem operasi induk untuk mencegah konflik pustaka.



Server

Mengonsolidasikan beberapa server logis ke dalam satu mesin fisik guna mengoptimalkan kapasitas.



Hardware

Menyajikan sumber daya komputasi fisik mentah menjadi perangkat virtual independen.

Fokus sesi: mengenali karakteristik masing-masing tipe, membandingkannya, lalu memilih solusi yang tepat.

Host menyediakan sumber daya; guest memakai bagian virtualnya

Guest VM 1 (Ubuntu Linux)

Sistem operasi tamu di dalam mesin virtual menjalankan aplikasi web dengan jatah vCPU dan vRAM yang diisolasi.

Guest VM 2 (Windows Server)

Sistem operasi tamu di dalam mesin virtual menjalankan basis data secara mandiri, tanpa memengaruhi VM 1.

Hypervisor (Virtual Machine Monitor / VMM)

Lapisan yang membuat dan mengelola VM serta membagi, mengisolasi, dan menjadwalkan alokasi CPU, RAM, penyimpanan, dan jaringan ke setiap VM.

Host Hardware (Server Fisik)

Komputer fisik penyedia sumber daya: prosesor multicore x86/ARM, memori ECC RAM fisik, SSD NVMe storage, dan antarmuka kartu jaringan (NIC).

Analogi Pengatur Lalu Lintas: Hypervisor bukan sumber daya itu sendiri, melainkan pengatur lalu lintas yang memastikan setiap guest mendapatkan hak akses hardware tanpa saling bertabrakan.

Hypervisor Type 1 berjalan langsung di atas hardware

Type 1 · Bare-metal

Langsung di atas bare-metal

Hypervisor bertindak sebagai sistem operasi khusus tanpa lapisan OS perantara. Mengelola CPU dan memori dengan overhead minimal serta performa tinggi.

Contoh industri: VMware ESXi, Proxmox VE, KVM enterprise, Microsoft Hyper-V Server.

Type 2 · Hosted

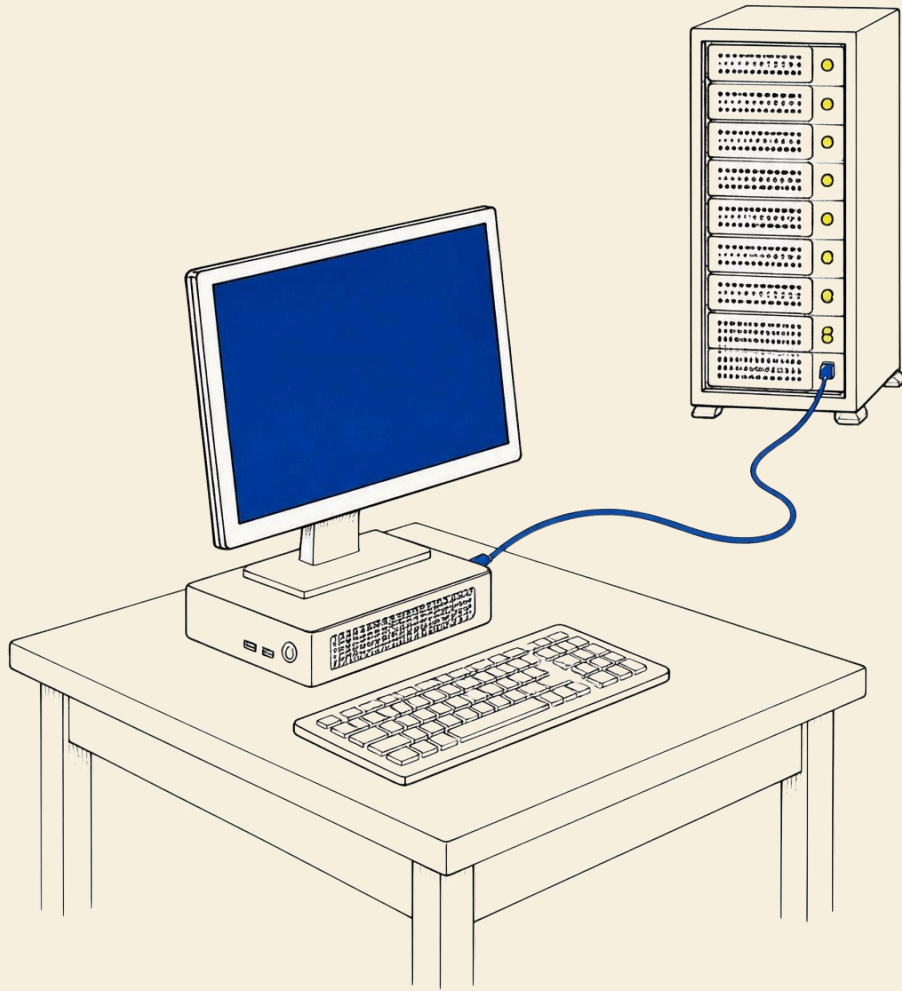
Berjalan di atas OS host

Hypervisor diinstal sebagai aplikasi reguler di atas Windows, macOS, atau Linux host. Bergantung pada penjadwal kernel sistem operasi utama.



Aktivitas kelas: Mahasiswa mengidentifikasi Oracle VirtualBox sebagai Type 2 karena membutuhkan OS induk untuk berjalan.

Desktop Virtualization memindahkan desktop ke pusat pengelolaan



Dalam **Desktop Virtualization**, seluruh lingkungan desktop pengguna dieksekusi di server pusat, mesin virtual, atau cloud data center, lalu diakses jarak jauh melalui jaringan.

Virtual Desktop Infrastructure (VDI): salah satu implementasi utama di mana setiap pengguna menerima desktop terisolasi yang di-host di pusat data, sementara perangkat lokal berfungsi sebagai media input-output (thin client).

Skenario Nyata

Laboratorium Komputer Kampus: Ratusan workstation mahasiswa dapat direset ke konfigurasi standar yang identik setiap sesi kuliah dimulai hanya dengan memperbarui satu image master di server.

Application Virtualization mengisolasi aplikasi dan dependensinya

Aplikasi dikemas dalam kapsul virtual yang membawakan runtime pustaka mandiri, memisahkannya dari modifikasi konfigurasi langsung pada sistem operasi host.

1	Aplikasi A (Sistem Legacy): Memerlukan pustaka Python 3.10 dan driver lama agar dapat berjalan stabil.	Python 3.10
2	Aplikasi B (Modul AI Baru): Mewajibkan framework modern berbasis Python 3.12 pada mesin yang sama.	Python 3.12

Tanpa virtualisasi aplikasi, instalasi dependensi global akan saling menimpa (dependency hell) dan merusak salah satu sistem. Lingkungan terisolasi memastikan keduanya berjalan damai berdampingan.

Manfaat

Distribusi aplikasi menjadi portabel dan konsisten: uji sekali pada lingkungan virtual, jalankan di mana saja tanpa khawatir perbedaan konfigurasi OS induk.

Container dan VM mengisolasi pada lapisan yang berbeda

Virtual machine (VM)

VM membawa sistem operasi guest bersama aplikasi dan perangkat keras virtual.

Container

Container umumnya mengemas aplikasi serta dependensinya, lalu berbagi kernel sistem operasi host melalui runtime.

Container biasanya lebih ringan daripada VM penuh.

Contoh teknologi terkait dalam sumber: Docker dan sandbox aplikasi.

Intinya: container tidak identik dengan VM.

Namespaces memisahkan pandangan; cgroups mengatur pemakaian

Linux Namespaces

Apa yang bisa dilihat oleh proses

Membatasi visibilitas sumber daya sehingga proses merasa memiliki sistem operasi sendiri.

PID: Nomor proses mandiri; proses utama container ber-PID 1. **NET:** Antarmuka jaringan, IP address, dan tabel routing independen. **MNT:** Pohon direktori filesystem terisolasi dari filesystem host. **USER & IPC:** Pemetaan ID user dan saluran inter-process communication.

Control Groups (cgroups)

Berapa banyak yang boleh dipakai

Menegakkan batas pemakaian kuota dan mengukur hardware host.

CPU quota: Membatasi persentase cycle clock agar tidak terjadi cpu starvation. **Memory limit:** Menetapkan batas RAM; memicu OOM-killer jika terlampaui. **Block I/O:** Mengatur kecepatan baca/tulis disk per kelompok proses.

Contoh: Satu server fisik dapat menampung puluhan container karena kernel secara adil membatasi alokasi via cgroups.

Virtualisasi juga membentuk jaringan dan perangkat I/O

1

vNIC / veth

Kartu jaringan virtual pada VM atau pasangan virtual ethernet container.

2

Virtual Switch / Bridge

Bridge perangkat lunak (seperti docker0) meneruskan frame data antar antarmuka.

3

Port Mapping / NAT

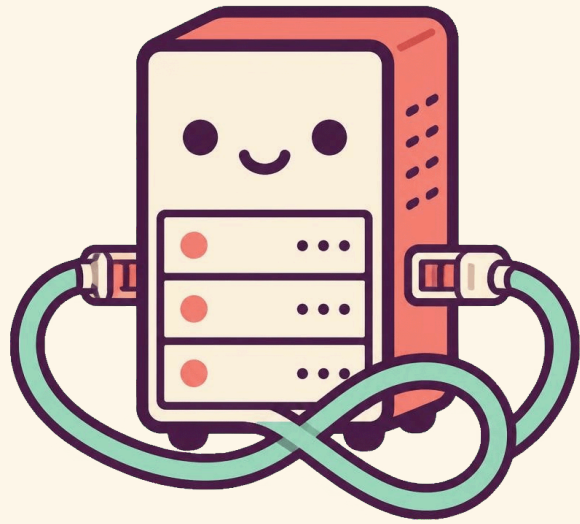
Translasi port host publik ke port internal container (misal: 0.0.0.0:8080 → 80).

4

NIC Fisik

Kartu jaringan fisik host menghantarkan paket lewat kabel ke switch data center.

Ingress / Reverse Proxy (misal Nginx) menerima HTTPS publik pada port 443, lalu meneruskannya ke port privat workload.



127.0.0.1 merujuk ke antarmuka loopback lokal di dalam namespace aktif.

Komunikasi antar-proses internal tanpa melintasi kabel jaringan fisik

Jika dua proses berjalan dalam namespace jaringan yang sama (seperti Pod Kubernetes), keduanya dapat bertukar data via localhost. Namun, jika berada di container terpisah dengan net namespace berbeda, 127.0.0.1 menunjuk ke diri masing-masing container, bukan container tetangga.

Storage sementara dan persistent punya siklus hidup berbeda

EPHEMERAL STORAGE

Layer sementara

File berada di writable layer container pada filesystem host lokal.

- **Siklus hidup:** File dapat hilang saat container dihapus atau environment dibangun ulang.
- Contoh: cache, unduhan sementara, dan berkas kerja yang dapat dibuat ulang.

PERSISTENT STORAGE

Volume & storage eksternal

Volume terpasang, object storage, database eksternal, atau network storage dapat mempertahankan data sesuai konfigurasi.

- **Mounted volume:** Direktori host atau managed volume.
- **Prinsip DevOps:** Pisahkan data stateful dari image container agar workload mudah di-scale.

Memory virtualization memetakan alamat virtual ke memori fisik

Setiap proses bekerja dalam ruang alamat virtual terisolasi yang diatur oleh Operating System dan Hardware MMU (Memory Management Unit).



Virtual Memory

Ilusi ruang alamat memori pribadi yang berurutan dan terisolasi untuk tiap proses.



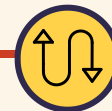
Paging

Pembagian memori menjadi blok-blok berukuran tetap (umumnya 4 KB) yang disebut frame & page.



Page Table

Struktur data indeks translasi pemetaan alamat virtual ke physical frame RAM.



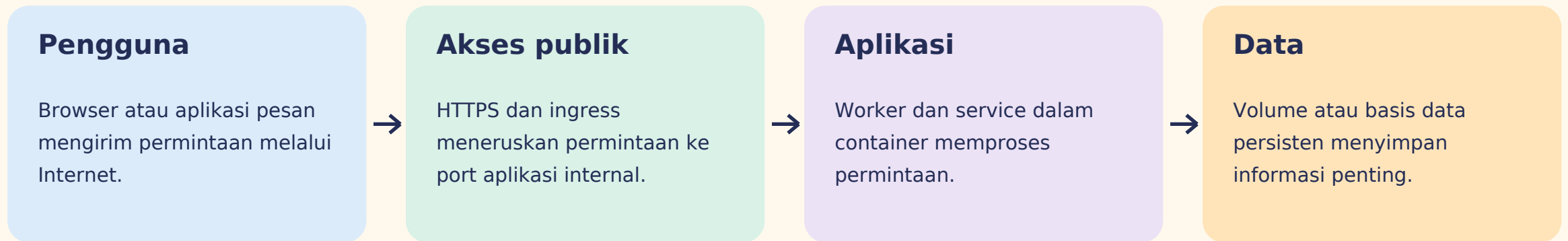
Swap Space

Area disk sekunder untuk menampung page tidak aktif saat memori fisik penuh.

Pembahasan mendalam mengenai algoritma penggantian page dan arsitektur TLB akan dilanjutkan pada Pertemuan 5.

Lapisan aplikasi cloud saling terhubung

Telusuri aliran permintaan, tempat proses berjalan, serta lokasi data disimpan.



Catatan: 127.0.0.1 hanya menghubungkan proses dalam namespace jaringan yang sama. GPU dipakai jika aplikasi memang menjalankan beban kerja yang memerlukannya.

Empat kelompok ahli, satu peta sistem

Bahas topik, lalu ajarkan hasilnya kepada anggota kelompok campuran.

A - Application Virtualization

Bandingkan VM dan container. Jelaskan fungsi namespaces dan cgroups.

B - Hardware Virtualization

Petakan host, hypervisor, guest, dan alokasi vCPU, RAM, disk, atau GPU.

C - Network Virtualization

Gambar jalur HTTPS ke aplikasi. Kapan loopback 127.0.0.1 berlaku?

D - Storage Virtualization

Bedakan file sementara dari data persisten; pilih lokasi riwayat pengguna.

Tahap 1: diskusi kelompok ahli | Tahap 2: kelompok campuran A + B + C + D